

PSYC 51.17: Models of Language and Communication - Week 4

Dimensionality Reduction for NLP

Lecture 13: PCA, t-SNE, and UMAP

PSYC 51.17: Models of Language and Communication - Week 4

Winter 2026

Winter 2026

Today's Lecture

1. **The Curse of Dimensionality**
2. **Principal Component Analysis (PCA)**
3. **t-SNE: Stochastic Neighbor Embedding**
4. **UMAP: Uniform Manifold Approximation**
5. **Comparison & Best Practices**
6. **Visualization Techniques**

Goal: Learn to visualize and explore high-dimensional embeddings

Winter 2026

The Curse of Dimensionality

The Problem:

Word Embeddings:

- Word2Vec: 300 dimensions
- GloVe: 300 dimensions
- FastText: 300 dimensions
- BERT: 768 dimensions
- GPT-3: 12,288 dimensions!

Challenges:

- Can't visualize 300D space
- Distances behave strangely

Concrete Example: Distance Concentration

```
1 import numpy as np
2
3 # Sample 1000 random points in
  different dims
4 for dim in [2, 10, 100, 300]:
5     points = np.random.randn(1000,
6                               dim)
7     dists = np.linalg.norm(points,
8                             axis=1)
9
10 print(f"Dim={dim}: mean=
11       {dists.mean():.2f}, "
```

3

The Curse of Dimensionality

```
9      f"std={dists.std():.2f}")
10
11     # Output:
12     # Dim=2: mean=1.26, std=0.52
13     # Dim=10: mean=3.08, std=0.49
14     # Dim=100: mean=9.95, std=0.50
15     # Dim=300: mean=17.3, std=0.50
```

All points cluster around the same distance from origin!

Winter 2026

Why Dimensionality Reduction?

Visualization:

- 2D/3D plots
- Explore semantic structure
- Identify clusters
- Discover patterns
- Quality assurance

Computation:

- Faster algorithms
- Less memory
- Enable real-time systems

Machine Learning:

- Reduce overfitting
- Feature selection
- Improve generalization
- Remove noise
- Combat curse of dimensionality

Interpretation:

- Understand relationships
- Identify important dimensions
- Communicate results

Principal Component Analysis (PCA)

The classic linear dimensionality reduction method

Intuition:

- Find directions of maximum variance
- Project data onto these directions
- First PC: most variance

PC1 captures most variation
PC2 captures remaining variation (orthogonal to PC1)

PCA: The Algorithm

Step-by-Step Worked Example:

Original Data (3D → 2D):

Word	x1	x2	x3
king	2.0	1.5	0.1
queen	1.8	1.6	0.2
man	1.0	0.5	0.1
woman	0.9	0.6	0.2

Eigenvalues (variance captured):

- $\lambda_1 = 0.85$ (85% variance)
- $\lambda_2 = 0.13$ (13% variance)
- $\lambda_3 = 0.02$ (2% variance)

Keep top 2 PCs → 98% variance retained!

7

- Step 1:** Center data (subtract mean)
- Step 2:** Compute covariance matrix C
- Step 3:** Find eigenvectors/eigenvalues

Projected Data (2D):

Word	PC1	PC2
------	-----	-----

PCA in Practice

```
1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # Assume we have word embeddings: shape (vocab_size, 300)
7 # For example, Word2Vec embeddings
8
9 # Standardize the data (optional but recommended)
10 scaler = StandardScaler()
11 embeddings_scaled = scaler.fit_transform(embeddings)
12
13 # Apply PCA
14 pca = PCA(n_components=2) # Reduce to 2D for visualization
```

Winter 2026

8

continued...

PCA in Practice

```
15 embeddings_2d = pca.fit_transform(embeddings_scaled)
16
17 # Check variance explained
18 print(f"Variance explained: {pca.explained_variance_ratio_}")
19 print(f"Total: {sum(pca.explained_variance_ratio_):.2%}")
20
21 # Visualize
22 plt.figure(figsize=(10, 8))
23 plt.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1], alpha=0.5)
24
25 # Annotate some words
26 words = ['king', 'queen', 'man', 'woman', 'cat', 'dog']
27 for word in words:
28     idx = word_to_idx[word]
```

Winter 2026

PCA in Practice

```
29 plt.annotate(word, (embeddings_2d[idx, 0], embeddings_2d[idx, 1]))
30
31 plt.xlabel('PC1')
32 plt.ylabel('PC2')
33 plt.title('Word Embeddings - PCA Projection')
34 plt.show()
```

Winter 2026

PCA Limitations

Assumes Linearity:

- Only captures linear relationships
- Word embeddings often have non-linear structure
- May miss important patterns

1 | Non-linear manifold → PCA struggles here!

Other Issues:

• **Variance $\neq$ Importance:**

- Preserves variance, not semantic structure
- Noisy dimensions may have high variance

• **Global focus:**

- May lose local structure
- Clusters can become blurred

• **Interpretability:**

- PCs are linear combinations

t-SNE: t-Distributed Stochastic Neighbor Embedding

Non-linear dimensionality reduction for visualization

Key Idea:

- Model similarity as probability
- High-D: Gaussian similarity
- Low-D: t-distribution

How it works:

1. Compute pairwise similarities in high-D:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|_2^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|_2^2 / 2\sigma_i^2)}$$

t-SNE: Key Concepts

Why t-distribution in low-D?

The Crowding Problem Illustrated:

```
1 High-D: 10 points can each have
2 9 equidistant neighbors
3
4 * * *
5 * * *
6 * * *
7
8 Low-D (2D): Can't fit 9
9 equidistant
10 neighbors around 1 point!
11
12 Solution: t-distribution has
13 heavier tails → allows
```

Perplexity Effect (Same Data!):

```
1 Perplexity = 5:
2 Tight, fragmented clusters
3 Good for fine structure
4
5 Perplexity = 30:
6 Balanced view
7 Standard choice
8
9 Perplexity = 100:
10 Merged clusters
11 More global view
```

t-SNE in Practice

```
1 from sklearn.manifold import TSNE
2 import matplotlib.pyplot as plt
3
4 # Apply t-SNE (can be slow for large datasets)
5 tsne = TSNE(
6     n_components=2, # 2D visualization
7     perplexity=30, # try 5-50
8     n_iter=2000, # iterations
9     random_state=42, # reproducibility
10    verbose=1 # show progress
11 )
12
13 embeddings_2d = tsne.fit_transform(embeddings)
14
15 # Visualize with labels
```


t-SNE in Practice

```
16 plt.figure(figsize=(12, 10))
17
18 # Color by semantic category (if available)
19 categories = ['animals', 'food', 'technology', ...]
20 colors = ['red', 'blue', 'green', ...]
21
22 for cat, color in zip(categories, colors):
23     mask = labels == cat
24     plt.scatter(
25         embeddings_2d[mask, 0],
26         embeddings_2d[mask, 1],
27         c=color,
28         label=cat,
29         alpha=0.6
30     )
```

Winter 2026

15

...continued...

t-SNE in Practice

```
31  
32 # Annotate some words  
33 for word, idx in word_to_idx.items():  
34     if word in important_words:  
35         plt.annotate(  
36             word,  
37             (embeddings_2d[idx, 0], embeddings_2d[idx, 1])  
38         )  
39  
40 plt.legend()  
41 plt.title('Word Embeddings - t-SNE Projection')  
42 plt.show()
```

Winter 2026

16

...continued

t-SNE Limitations and Caveats

1. Slow: $O(n^2)$ complexity

```
1 # Timing comparison
2 n=1000: ~10 seconds
3 n=5000: ~4 minutes
4 n=10000: ~15 minutes
```

2. Non-deterministic:

```
1 tsne1 =
  TSNE(random_state=42)
2 tsne2 =
  TSNE(random_state=123)
3 # Different layouts!
```

4. Interpretation Pitfalls:

```
1 WRONG interpretations:
2 "Cluster A is bigger than
3 B"
4 (sizes are arbitrary)
5
6 "A and B are far apart"
7 (global distances not
8 preserved)
9
10 "This dimension means X"
    (axes have no meaning)
```

UMAP: Uniform Manifold Approximation & Projection

Modern alternative to t-SNE (2018)

Key Advantages over t-SNE:

- ($O(n \log n)$ vs $O(n^2)$)
- Preserves local and global structure
- Can transform new points
- Theoretically grounded (topology)
- Scales to millions of points
- More robust hyperparameters

When to Use:

- Large datasets ($>10^4$ points)

How it Works:

1. Construct fuzzy topological representation of high-D data
2. Find low-D representation with similar topology
3. Use Riemannian geometry
4. Optimize cross-entropy loss

Main Hyperparameters:

1. `n_neighbors` (15):

UMAP in Practice

```
1 import umap
2 import matplotlib.pyplot as plt
3
4 # Apply UMAP
5 reducer = umap.UMAP(
6     n_components=2, # 2D visualization
7     n_neighbors=15, # local/global balance (try 5-50)
8     min_dist=0.1, # cluster tightness (try 0.0-0.99)
9     metric='cosine', # good for word embeddings
10    random_state=42
11 )
12
13 embeddings_2d = reducer.fit_transform(embeddings)
14
15 # Can also transform new points! (unlike t-SNE)
16 new_embeddings_2d = reducer.transform(new_embeddings)
```

Winter 2026

19

continued...

UMAP in Practice

```
17
18 # Visualize
19 plt.figure(figsize=(12, 10))
20 scatter = plt.scatter(
21     embeddings_2d[:, 0],
22     embeddings_2d[:, 1],
23     c=cluster_labels, # color by cluster
24     cmap='Spectral',
25     s=5,
26     alpha=0.6
27 )
28 plt.colorbar(scatter)
29
30 # Annotate
31 for word in important_words:
32     idx = word_to_idx[word]
```

Winter 2026

20

...continued...

UMAP in Practice

```
33 plt.annotate(  
34 word,  
35 (embeddings_2d[idx, 0], embeddings_2d[idx, 1]),  
36 fontsize=12  
37 )  
38  
39 plt.title('Word Embeddings - UMAP Projection')  
40 plt.show()
```

Winter 2026

PCA vs. t-SNE vs. UMAP

Feature	PCA	t-SNE	UMAP
Speed	Very Fast	Slow	Fast
Scalability	Excellent	Poor (<10k)	Excellent
Global structure	Yes	No	Yes
Local structure	Partial	Yes	Yes
Deterministic	Yes	No	Partial
Out-of-sample	Yes	No	Yes

Concrete Timing Comparison (10,000 word embeddings):

```
1 | # PCA: 0.3 seconds
```

Best Practice Workflow

```
1 | # Step 1: PCA to 50D (fast, removes noise)
2 | X_50d = PCA(n_components=50).fit_transform(X)
```

- t-SNE perplexity: 5, 10, 30, 50
- UMAP n_neighbors: 5, 15, 30, 50
- Different views reveal different structure

3. **Color intelligently:**

- By semantic category
- By cluster assignment
- By frequency (size)
- By POS tag, domain, etc.

4. **Interactive visualization:**

- Use plotly, bokeh for interactivity

Interactive Visualization

```
1 import plotly.express as px
2 import plotly.graph_objects as go
3 import pandas as pd
4
5 # Prepare data
6 df = pd.DataFrame({
7     'x': embeddings_2d[:, 0],
8     'y': embeddings_2d[:, 1],
9     'word': words,
10    'category': categories,
11    'frequency': frequencies
12 })
13
14 # Create interactive plot with plotly
15 fig = px.scatter(
16     df,
17     x='x',
```

Interactive Visualization

```
18     y='y',
19     color='category',
20     size='frequency',
21     hover_data=['word', 'frequency'],
22     title='Word Embeddings (UMAP)',
23     width=1000,
24     height=800
25 )
26
27 # Add text annotations for important words
28 for word in important_words:
29     row = df[df['word'] == word].iloc[0]
30     fig.add_annotation(
31         x=row['x'],
32         y=row['y'],
33         text=word,
34         showarrow=False,
```

25

Winter 2026

...continued...

Interactive Visualization

```
35     font=dict(size=12)
36 )
37
38 # Show
39 fig.show()
40
41 # Can also save as HTML
42 fig.write_html('embeddings_viz.html')
```

Winter 2026

Applications in NLP

1. Embedding Quality Check:

```
1 # Quick sanity check
2 words = ['dog', 'cat', 'fish', #
3         'car', 'bus', 'train'] #
4         vehicles
5 # If good embeddings: two
6 # clusters!
7 # If bad: random scatter
```

2. Finding Polysemous Words:

```
1 "apple" appears in TWO clusters:
2 - Near: orange, banana, fruit
3 - Near: microsoft, google, tech
```

4. Model Layer Comparison:

```
1 # Extract embeddings from
2 # different layers
3 layer_1 = get_bert_layer(1) #
4 Syntax
5 layer_6 = get_bert_layer(6) #
6 Semantics
7 layer_12 = get_bert_layer(12) #
8 Task-specific
9
10 # Visualize each: different
11 # structure!
```

27

5. Document Clustering:

```
1 # Visualize document embeddings
```

Case Study: Visualizing BERT Layers

Question: What do different BERT layers capture?

Approach:

1. Extract embeddings from each layer
2. Apply UMAP to each layer separately
3. Visualize and compare

Typical Findings:

- **Layer 0-2:** Syntactic (POS tags cluster)
- **Layer 3-8:** Semantic (meaning clusters)
- **Layer 9-12:** Task-specific

Insights:

```
1 from transformers import BertModel, BertTokenizer
2
3 model = BertModel.from_pretrained('bert-base-uncased', output_hidden_states=True)
4 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
5
6 # Get embeddings from all layers
7 tokens = tokenizer(sentences, return_tensors='pt', padding=True)
8 outputs = model(**tokens)
```

28

continued...

Case Study: Visualizing BERT Layers

```
9
10 # outputs.hidden_states: tuple of 13 tensors
11 # (embedding layer + 12 transformer layers)
12
13 for layer_idx in range(13):
14     layer_embeddings = outputs.hidden_states[layer_idx]
15
16     # Average over sequence length
```

Winter 2026

Case Study: Visualizing BERT Layers

```
17 avg_embeddings = layer_embeddings.mean(dim=1).numpy()  
18  
19 # Apply UMAP  
20 reducer = umap.UMAP(n_components=2)  
21 viz_2d = reducer.fit_transform(avg_embeddings)  
22  
23 # Plot  
24 plt.figure()
```

Winter 2026

Case Study: Visualizing BERT Layers

```
25 plt.scatter(viz_2d[:, 0], viz_2d[:, 1], c=pos_tags)
26 plt.title(f'Layer {layer_idx}')
27 plt.show()
```

Reference: Jawahar et al. (2019). "What Does BERT Learn about the Structure of Language?"

Winter 2026

Discussion Question

What does a 2D visualization actually tell us about 300D space?

Consider:

- We compress 300 dimensions into 2
- Massive information loss (>99%)
- Different methods show different views
- Hyperparameters change the story

What we CAN conclude:

- Rough semantic groupings
- Relative proximities
- Cluster existence

What we CANNOT conclude:

- Exact distances
- True high-D structure
- Cluster sizes (t-SNE)

- Remove extreme outliers

3. Try multiple settings:

- Run with different hyperparameters
- Compare results
- Don't cherry-pick!

4. Use color wisely:

- Semantic categories
- Continuous values (frequency, sentiment)
- Consider colorblind-friendly palettes

3. **t-SNE:**

- Non-linear, preserves local structure
- Beautiful visualizations but slow
- Sensitive to hyperparameters

4. **UMAP:**

- Fast, scalable, balanced local+global
- Best default choice for NLP

Applications:

- Jawahar et al. (2019). "What Does BERT Learn about the Structure of Language?"
- Coenen et al. (2019). "Visualizing and Measuring the Geometry of BERT"

Guides & Tutorials:

- Wattenberg et al. (2016). "How to Use t-SNE Effectively" (Distill)
- UMAP documentation: <https://umap->

Questions?

Next Lecture:

Cognitive Models of Semantic Representation

How do humans represent meaning?

Winter 2026